

第4章 Storm | 开卷速查版

本章一句话：Storm 是一个分布式实时流式计算平台，适合处理持续到来的数据流，例如日志实时统计、异常检测、热点分析。它的程序不是跑完就结束的 Job，而是一个长期运行的 **Topology**。☁️

常考题型：Storm vs MapReduce / queue+worker；解释 Topology、Spout、Bolt、Tuple、Stream、Grouping；给一个数据流设计几个 Bolt；判断用 **shuffleGrouping** 还是 **fieldsGrouping**；解释 Storm 系统架构和数据流转过程。

翻页关键词：为什么需要 Storm → 和其他平台对比 → 计算模型 → Grouping → Bolt 链路设计 → 系统架构 → 并发层次 → 数据流程 → 容错/Ack。

1. 为什么需要 Storm

传统 **MapReduce** 适合离线批处理，数据通常先落到 HDFS，再启动作业分析，延迟比较高。现实里有很多任务要求数据一来就处理，比如日志 PV/UV、异常监控、访问热点、实时推荐、实时告警。

PPT里说 Storm 是 **分布式实时流式计算平台**，具有分布式、水平扩展、自动容错、实时、流式等特点；实时性来自数据主要在内存中流动，不像 MapReduce 那样频繁写磁盘。

考试写法：Storm 用于解决实时流式计算问题。它可以对不断到来的数据进行低延迟处理，并通过增加机器和提高并发度实现水平扩展，同时自动处理进程、机器和网络异常，因此适合实时统计、异常检测和流式数据处理场景。

2. Storm vs MapReduce

对比点	MapReduce	Storm
任务形态	一个 Job 跑完结束	Topology 常驻运行
数据输入	静态批量数据	连续不断的数据流
处理方式	批处理	来一条处理一条
中间结果	大量落盘 / Shuffle	内存中传递，延迟低
模型	Map + Reduce 两阶段	DAG，多阶段组合
适合	离线统计、批处理	实时计算、流式处理

PPT里明确对比：Storm 相比 MapReduce 是**常驻运行、流式处理、实时处理、DAG模型**。

考试写法：MapReduce 面向离线批处理，作业通常处理静态数据集，运行结束后退出；Storm 面向实时流式处理，Topology 会长期运行，数据不断流入、处理和流出。Storm 中间数据主要在内存中传递，延迟更低，并且可以用 DAG 组合多个处理阶段。

3. Storm vs queue + worker 系统

传统做实时处理，也可以用 **消息队列 + worker**。比如 Kafka / queue 存消息，多个 worker 消费消息。

但这种方式要自己维护很多事情：哪个 worker 消费哪个 queue、worker 挂了怎么办、消息失败是否重发、如何扩容、如何重新分配任务。

Storm 的优势是：

维护简单： 不需要手动维护 queue 和 worker 的对应关系。**扩展简单：** 加机器、提高并发、重新提交 topology 即可。**自动容错：** 进程、机器、网络异常可以自动处理，消息可以重发。

PPT中也把 Storm 相比 queue+worker 的优势列为维护简单、扩展简单、自动容错。

考试写法： Storm 相比手写 queue+worker 系统，屏蔽了队列与 worker 绑定、任务调度、失败恢复和消息重发等复杂细节。用户只需要描述流式计算逻辑，Storm 负责并发执行、扩容和容错。

4. Storm 的典型应用场景

Storm 适合两类任务：**异步流式处理** 和 **同步请求应答 DRPC**。

异步流式处理： 异常监测、日志处理入库、实时 PV/UV 统计、访问热点统计、实时风控。

同步请求应答： DRPC，即 Distributed RPC，把一次请求也当成流式任务来并行处理，比如实时图片处理、实时网页处理。

PPT中列出 Storm 典型应用场景：异常监测、实时分析统计、日志 PV/UV、访问热点统计，以及 DRPC、实时图片/网页处理。

考试写法： Storm 适合处理连续到来的实时数据流，例如异常监测、日志实时统计、访问热点分析等；也可以通过 DRPC 支持同步请求应答，把 RPC 请求作为输入流并行处理后返回结果。

5. Storm 计算模型总览

Storm 程序的核心叫 **Topology**。它是一个长期运行的 DAG，里面由 **Spout**、**Bolt**、**Stream**、**Tuple**、**Grouping** 组成。

可以画成：

```
Spout
|
| Stream<Tuple>
v
Bolt 1
|
v
Bolt 2
|
v
Bolt 3 / Sink
```

Topology： 一直运行的 Job，是整个流式计算任务。**Spout：** 数据源，负责产生 tuple。**Bolt：** 数据处理节点，负责处理 tuple。**Stream：** tuple 流动的管道。**Tuple：** Storm 处理数据的基本单位。**Grouping：** 决

定 tuple 发给下游 Bolt 的哪个并发实例。

PPT中定义 Topology 是一直运行的 Job，并且其 DAG 描述包括 Spout、Bolt、Stream、Tuple、Grouping。

6. Tuple 和 Stream

Tuple 是 Storm 处理数据的基本单位，可以理解成一条记录。它有固定字段名和字段顺序，也就是 schema。

比如日志 tuple 可以是：

```
<user_id, url, timestamp, ip>
```

Stream 是 tuple 的管道。一个 stream 里的 tuple 具有固定 schema。每个 Spout / Bolt 都有默认 stream，名字通常叫 default；一个 Spout 或 Bolt 也可以输出多个 stream。

考试写法： Tuple 是 Storm 中流动的数据基本单位，由字段名和字段顺序定义 schema；Stream 是 Tuple 的管道，一个 Stream 中的 Tuple 具有固定 schema。Spout 和 Bolt 可以接收或产生一个或多个 Stream。

7. Spout

Spout 是数据源，负责从外部系统中读取数据并发出 tuple。常见来源包括 Kafka、消息队列、日志系统等。

PPT里强调：Spout 通常从外部数据源如 Kafka 中“拉”数据，而不是被动等待数据推来。这样 Storm 不需要自己做源数据持久化缓存，也可以做流量控制，并且能和 Ack 机制配合保证可靠性。

核心函数是：

```
nextTuple()
```

它会不断被 Storm 回调，用来产生新的 tuple。

考试写法： Spout 是 Storm 的数据源，通常从 Kafka 等外部系统拉取数据并生成 Tuple。它的核心回调函数是 nextTuple()。拉取模式便于进行流量控制，并可结合 Ack 机制实现消息可靠处理。

8. Bolt

Bolt 是 Storm 的处理节点，负责接收 tuple、执行计算逻辑、再输出新的 tuple。

Bolt 可以做很多事：

```
解析日志  
过滤无效数据  
字段转换  
按 key 统计  
窗口聚合
```

写入数据库
触发告警

核心函数是：

```
execute(Tuple input)
```

考试写法： Bolt 是 Storm 中执行具体业务逻辑的组件。它接收输入 Tuple，在 execute() 方法中完成解析、过滤、聚合、写库等处理，并可以继续向下游发射新的 Tuple。

9. Grouping 策略

Grouping 决定一个 tuple 从上游发送到下游 Bolt 的哪个并发实例。

重点掌握两个：**shuffleGrouping** 和 **fieldsGrouping**。

Grouping	含义	适合场景
shuffleGrouping	随机分发	无状态处理、负载均衡
fieldsGrouping	按字段 hash 分发	同一个 key 必须到同一个 Bolt
allGrouping	广播到全部实例	规则广播、配置广播
directGrouping	指定某个 task	精确控制路由
customGrouping	自定义分发	特殊路由逻辑

shuffleGrouping： 如果每条 tuple 可以独立处理，不需要记住同一个 key 的历史，就用 shuffle。比如日志清洗、格式转换、过滤。

fieldsGrouping： 如果同一个 key 的数据必须进入同一个 Bolt 实例，就用 fields。比如 WordCount 中同一个 word 的计数必须在同一个 Bolt 里累加；用户行为统计中同一个 user_id 要送到同一个 Bolt。

PPT中定义 Grouping 是路由策略，即 tuple 发往下游 Bolt 的多个并发实例中的哪一个，并列出了 shuffle、fields、direct、all、custom 等策略。

考试写法： Grouping 是 Storm 中 tuple 的路由策略。shuffleGrouping 随机分发 tuple，适合无状态处理和负载均衡；fieldsGrouping 按指定字段 hash 分发，保证相同字段值的 tuple 进入同一个下游 Bolt 实例，适合按 key 聚合统计。

10. 给一个流，怎么判断链条里有几个 Bolt

考试很可能给一个场景，让你设计 Storm 拓扑。方法是：**按处理阶段拆 Bolt，不按数据条数拆 Bolt。**

通用模板：

Spout

- > 解析 Bolt
- > 过滤 Bolt
- > 转换/清洗 Bolt
- > 按 key 聚合 Bolt
- > 入库/告警 Bolt

判断规则：

第一步：找数据源。 来自 Kafka / 日志 / 消息队列，就是 Spout。

第二步：找独立处理阶段。 解析、过滤、转换这些可以拆成不同 Bolt，也可以合并，看题目要求。

第三步：看有没有按 key 聚合。 只要出现“按用户统计、按 URL 统计、按单词统计、按地区统计”，就需要一个聚合 Bolt，并且上游到这个 Bolt 通常用 **fieldsGrouping(key字段)**。

第四步：看最后输出到哪里。 写数据库、写 HBase、发告警、写 Kafka，可以是 Sink Bolt。

11. Bolt 链路设计模板

场景 A：实时 WordCount

```
SentenceSpout
-> SplitBolt
-> CountBolt
-> Print/StoreBolt
```

连接方式：

```
SentenceSpout --shuffle--> SplitBolt
SplitBolt --fields(word)--> CountBolt
CountBolt --shuffle--> Print/StoreBolt
```

解释：

Spout → SplitBolt 用 shuffle： 每句话独立拆词，随机分发即可。 **SplitBolt → CountBolt 用 fields(word)：** 同一个 word 必须进入同一个 CountBolt，否则计数会分散。 **CountBolt → StoreBolt 用 shuffle：** 写库可以负载均衡，通常随机即可。

场景 B：日志实时 PV/UV

```
KafkaSpout
-> ParseLogBolt
-> FilterBolt
```

```
-> PVUVCountBolt
-> StoreBolt
```

连接方式：

```
KafkaSpout --shuffle--> ParseLogBolt
ParseLogBolt --shuffle--> FilterBolt
FilterBolt --fields(url 或 user_id)--> PVUVCountBolt
PVUVCountBolt --shuffle--> StoreBolt
```

解释：

PV 按 URL 统计，就 `fields(url)`；UV 按 URL + user_id 去重，可能要按 URL 或组合 key 分区。

场景 C：异常检测

```
MetricSpout
-> ParseMetricBolt
-> RuleCheckBolt
-> AlertBolt
-> StoreBolt
```

连接方式：

无状态解析用 shuffle；如果要按机器、服务、用户维持历史状态判断趋势，就用 `fields(host_id)` 或 `fields(service_id)`。

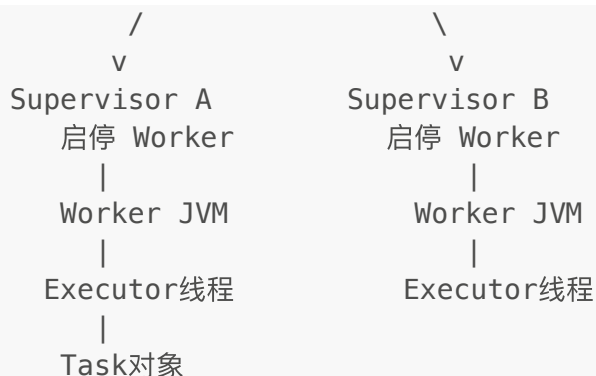
考试写法：设计 Storm 拓扑时，先确定数据源 Spout，再按处理阶段划分 Bolt。无状态处理阶段之间使用 shuffleGrouping 来均衡负载；需要按某个 key 聚合、计数或维护状态时，使用 fieldsGrouping，保证相同 key 的 tuple 被发送到同一个 Bolt 实例。

12. Storm 系统架构总览

Storm 架构主要有 **Nimbus**、**Supervisor**、**Worker**、**Executor**、**Task**、**ZooKeeper**。

可以画成：





PPT架构页列出：Nimbus 管理集群和调度 topology；Supervisor 启停 worker；Worker 是 JVM 进程，是资源分配单位；Executor 是实际干活的线程；ZooKeeper 存储调度信息和心跳。

13. Nimbus

Nimbus 是 Storm 的主控节点，类似集群管理者。

主要职责：

1. 接收 topology 提交。
2. 调度 topology。
3. 把调度信息写入 ZooKeeper。
4. 检查 Supervisor / Worker 心跳。
5. 处理 submit、kill、rebalance 请求。
6. 提供集群和 topology 状态查询接口。
7. 提供 topology jar 包上传下载服务。

PPT里说 Nimbus 负责调度 topology、写调度信息到 ZK、检查心跳、处理 submit/kill/rebalance、提供状态查询和 jar 包上传下载服务。

考试写法： Nimbus 是 Storm 的主控组件，负责接收和调度 Topology，将调度信息写入 ZooKeeper，监控 Supervisor 和 Worker 的心跳，并处理提交、终止和重新平衡等请求。

14. Supervisor

Supervisor 运行在每台工作节点上，负责根据 ZooKeeper 中的调度信息启动或停止本机上的 Worker。

主要职责：

1. 从 ZooKeeper 读取调度信息。
2. 下载并解压 topology jar 包。
3. 启动 Worker JVM 进程。
4. 检查本机 Worker 心跳。
5. Worker 异常时重启。
6. 定期向 ZooKeeper 写 Supervisor 心跳。

PPT里说 Supervisor 根据 ZK 调度信息启动 worker，下载 topology jar 包，启动 worker JVM，检查 worker 本地心跳并处理异常。

考试写法： Supervisor 是运行在工作节点上的守护进程，负责根据 ZooKeeper 中的调度信息启动和停止本机 Worker 进程，并监控 Worker 的本地心跳，发现异常时进行重启或等待重新调度。

15. Worker

Worker 是一个 JVM 进程，是 Storm 中资源分配的基本单位。一个 Worker 会运行多个 Executor。

主要职责：

1. 启动 Executor。
2. 负责 Worker 之间的数据传输。
3. 建立网络连接。
4. 使用收发线程传输 tuple。
5. 定期更新本地和 ZooKeeper 心跳。

PPT明确说 Worker 是一个 JVM 进程，是资源分配单位，负责启动 Executor 和数据传输。

考试写法： Worker 是运行在 Supervisor 节点上的 JVM 进程，是 Storm 的资源分配单位。它负责启动 Executor，并负责不同 Worker 之间的网络连接和 Tuple 传输。

16. Executor

Executor 是真正执行 Spout 或 Bolt 逻辑的线程。

PPT里说 Executor 会创建 Spout / Bolt 对象，并运行两个关键线程：执行线程负责调用 `nextTuple()` 或 `execute()`，传输线程负责把新产生的 tuple 放到 Worker 传输队列。

Executor 内部有两个关键队列：

Receive Queue： 收到的 tuple 先放这里，执行线程从这里取 tuple 处理。**Transfer Queue：** 执行线程 emit 的 tuple 放这里，之后交给 Worker 传输。

考试写法： Executor 是 Storm 中实际执行 Spout 或 Bolt 的线程。它包含执行线程和传输线程：执行线程调用 `nextTuple()` 或 `execute()` 处理数据，传输线程将新产生的 Tuple 放入 Worker 的传输队列。Executor 内部通常维护 receive queue 和 transfer queue。

17. Task

Task 是 Spout 或 Bolt 的对象实例。可以理解成执行逻辑的具体对象。

并发层级要背：

```
Cluster
-> Supervisor: 节点
```



```
-> Worker: 进程  
-> Executor: 线程  
-> Task: 对象
```

PPT也给出并发模型层次：Cluster、Supervisor-node、Worker-process、Executor-thread、Task-object。

考试写法：Storm 的并发层次是 Cluster → Supervisor → Worker → Executor → Task。其中 Supervisor 对应节点，Worker 是 JVM 进程，Executor 是线程，Task 是具体的 Spout/Bolt 对象实例。

18. ZooKeeper

ZooKeeper 是 Storm 的可靠状态存储和协调组件。

主要存：

1. Supervisor 心跳
2. Worker 心跳
3. 调度信息
4. 错误信息
5. 集群元数据

PPT里说 Zookeeper 存储心跳、调度信息、错误信息等元数据，被 Storm 当成高可用 KV 使用。

考试写法：ZooKeeper 在 Storm 中用于存储心跳、调度信息、错误信息等元数据，起到可靠状态存储和协调作用。Nimbus 将调度信息写入 ZooKeeper，Supervisor 根据这些信息启动或停止 Worker。

19. Storm 数据流程

这个题如果考系统架构，可能会问 tuple 在内部怎么流转。按 PPT 数据流程背：

1. Executor 执行 `spout.nextTuple()` 或 `bolt.execute()`。
2. 调用 `emit` 产生新的 tuple。
3. tuple 放入 Executor 的 transfer queue。
4. Executor 的 transfer 线程把 tuple 放到 Worker transfer queue。
5. Worker transfer 线程把 tuple 发送到远程 Worker。
6. 远程 Worker receive 线程从网络接收 tuple。
7. tuple 被放入目标 Executor 的 receive queue。
8. 目标 Executor 从 receive queue 取出 tuple。
9. 调用 `bolt.execute()` 继续处理。

PPT中几乎逐条给出该数据流程。

考试写法：Storm 中，Executor 调用 Spout 的 `nextTuple()` 或 Bolt 的 `execute()` 处理数据并 emit 新 Tuple。新 Tuple 先进入 Executor 的 transfer queue，再由传输线程放入 Worker transfer queue。若目标在远程节点，Worker 通过网络发送给远程 Worker；接收端将 Tuple 放入目标 Executor 的 receive queue，最后由 Executor 取出并调用 Bolt 的 `execute()` 处理。

20. Storm 容错：控制层容错

Storm 容错分成两层：**控制层容错** 和 **数据层容错**。

控制层负责组件故障：

Nimbus down：重启即可，不影响已经运行的任务。因为 Worker 的运行不依赖 Nimbus 持续存在。但 Nimbus 挂掉期间，如果 Worker 也挂了，就不能重新调度。

Supervisor down：该机器上的 Worker 都会失效，ZooKeeper 心跳丢失后，任务会被重新调度到其他机器。

Worker down：Supervisor 会重启 Worker；如果一直失败，Nimbus 会重新调度到其他节点。

PPT中把这些分别列为 Nimbus down、Supervisor down、Worker down 的处理方式。

考试写法：Storm 的控制层容错负责处理组件故障。Nimbus 挂掉后重启即可，正在运行的 Worker 通常不受影响；Supervisor 挂掉会导致本机 Worker 失效，任务可被重新调度到其他节点；Worker 挂掉时 Supervisor 会尝试重启，持续失败则由 Nimbus 重新调度。

21. Storm 容错：数据层 Ack 机制

数据层容错负责保证 tuple 被处理。核心是 **Ack 机制**。

流程：

1. Spout 发出 tuple，并附带 message id。
2. Bolt 处理 tuple，并可能产生子 tuple。
3. 子 tuple 继续被下游 Bolt 处理。
4. 所有子路径都处理完成后，acker 汇总成功信息。
5. acker 通知 Spout ack。
6. 如果某条 tuple 未 ack 或 timeout，Spout 重新发送。

PPT里说 Ack 机制用于确保每条 tuple 最终被处理，流程是 Spout 发出带 id 的 tuple，Bolt 处理并产生子 tuple，所有子路径完成后 ack，未 ack 或超时则 Spout 重发。

考试写法：Storm 通过 Ack 机制实现数据级容错。Spout 发出 Tuple 时带有消息 id，Bolt 处理后产生的子 Tuple 会被追踪。当该 Tuple 派生出的所有处理路径都完成时，acker 通知 Spout 成功；如果超时或失败，Spout 会重新发送该 Tuple。

22. Storm Java API 简记

这部分如果不考实现，可以只记两个核心回调函数：

Spout：

```
nextTuple()
```

持续拉取或产生数据。

Bolt:

```
execute(Tuple input)
```

处理输入 tuple。

PPT中也明确写：Spout 核心是 `nextTuple()`，Bolt 核心是 `execute(Tuple input)`。

考试写法：Storm API 中，Spout 的核心函数是 `nextTuple()`，用于不断产生 Tuple；Bolt 的核心函数是 `execute(Tuple input)`，用于处理输入 Tuple 并可继续向下游发射结果。

23. Storm 的状态、窗口和局限

PPT里提到 Storm 早期没有 window，需要用户手写；后期有 Windowed Bolt，支持 sliding / tumbling window。但整体上 Storm 的状态能力较弱，很多状态需要用户自己维护，语义也比较弱，没有原生 event time。

这部分和后面的 Flink 对比很重要：

Storm: 低延迟真流处理，但状态、窗口、exactly-once、event time 支持较弱。**Flink:** 原生流处理，同时支持 event time、watermark、managed state、checkpoint、exactly-once。

考试写法：Storm 适合低延迟实时流处理，但早期窗口和状态支持较弱，复杂状态通常需要用户自行维护；相比 Flink，Storm 对 event time、watermark、状态管理和 exactly-once 的支持不够完善。

本章高频简答模板

1. Storm 是什么？ Storm 是分布式实时流式计算平台。它通过长期运行的 Topology 处理不断到来的数据流，具有水平扩展、低延迟和自动容错能力，适合实时统计、异常检测和日志流处理。

2. Storm 和 MapReduce 的区别？ MapReduce 面向离线批处理，处理静态数据集，作业运行结束后退出；Storm 面向实时流式处理，Topology 常驻运行，数据来一条处理一条，中间数据主要在内存中传递，因此延迟更低。

3. Storm 的计算模型包括什么？ Storm 的计算模型包括 Topology、Spout、Bolt、Stream、Tuple 和 Grouping。Topology 是长期运行的 DAG；Spout 是数据源；Bolt 是处理节点；Tuple 是基本数据单位；Stream 是 Tuple 管道；Grouping 决定 Tuple 如何路由到下游 Bolt 实例。

4. shuffleGrouping 和 fieldsGrouping 的区别？ shuffleGrouping 随机分发 Tuple，适合无状态处理和负载均衡；fieldsGrouping 按指定字段 hash 分发，保证相同字段值进入同一个 Bolt 实例，适合按 key 统计、聚合或维护状态。

5. Storm 架构组件有哪些？ Storm 主要由 Nimbus、Supervisor、Worker、Executor、Task 和 ZooKeeper 组成。Nimbus 负责任务调度，Supervisor 负责启动 Worker，Worker 是 JVM 进程，Executor 是线程，Task 是具体对象实例，ZooKeeper 存储心跳和调度信息。

6. Storm 如何容错？ Storm 控制层通过 Nimbus、Supervisor 和 ZooKeeper 处理组件故障；数据层通过 Ack 机制追踪 Tuple 的处理过程。当 Tuple 的所有子路径都完成时通知 Spout 成功；若超时或失败，Spout 重新发送该 Tuple。

本章易混点

Topology 不是普通 Job。 MapReduce Job 会结束，Storm Topology 通常一直运行。

Spout 是数据源，Bolt 是处理逻辑。 不要把 Spout 写成计算节点。Spout 主要负责产生 tuple。

shuffleGrouping 不是 Shuffle。 Storm 的 shuffleGrouping 是随机路由策略；MapReduce 的 Shuffle 是 Map 到 Reduce 的重分区、排序、分组过程。

fieldsGrouping 用在按 key 聚合。 只要需要同一个 word/user/url 到同一个 Bolt，就用 fieldsGrouping。

Worker / Executor / Task 层级要分清。 Worker 是进程，Executor 是线程，Task 是对象实例。

本章看到题目怎么写

看到“**Storm vs MapReduce**”：写批处理 vs 实时流；Job 结束 vs Topology 常驻；磁盘/HDFS vs 内存流动。

看到“**设计 Storm 拓扑**”：先写 Spout，再按阶段拆 Bolt；无状态阶段用 shuffle；按 key 聚合用 fields。

看到“**WordCount Storm 链路**”：SentenceSpout -> SplitBolt -> CountBolt -> StoreBolt，Split 前后用 shuffle，进入 Count 用 fields(word)。

看到“**系统架构**”：写 Nimbus、Supervisor、Worker、Executor、Task、ZooKeeper，并写并发层级。

看到“**容错**”：写控制层组件故障 + 数据层 Ack 机制。